

LAPORAN PENGUJIAN PERANGKAT LUNAK

Pengujian Otomatis Modul Login & Register Menggunakan Selenium WebDriver, Stub Layer, dan CI/CD Pipeline (GitHub Actions)

Mata Kuliah	: Pengujian Perangkat Lunak (Pertemuan 15)
Topik / Tugas	: Quiz Pengujian Piranti Lunak — Modul Login & Register
Repository Target	: https://github.com/hermanka/quiz-pengupil
Tools Pengujian	: Selenium WebDriver, Python unittest, PHP CLI
CI/CD Engine	: GitHub Actions (MySQL 8.0 Container)
Status Testing	: 100% PASSED (8/8 Test Cases Berhasil)

INFORMASI PENYERAHAN TUGAS

Dokumen laporan ini disusun sebagai bukti penyelesaian tugas praktikum pengujian perangkat lunak mencakup perancangan test case, implementasi stub & driver, eksekusi Selenium automation script, petunjuk jalannya testing untuk penguji, serta otomatisasi pipeline CI/CD di GitHub Actions.

BAB I: ANALISIS SISTEM & REFACTORING MODUL TARGET

1.1 Deskripsi Modul

Pengujian dilakukan terhadap aplikasi berbasis web PHP-MySQL dari repositori target **quiz-pengupil**. Modul yang diuji terdiri dari dua komponen utama:

- **Modul Login (login.php):** Menangani autentikasi pengguna berdasarkan verifikasi username dan password terenkripsi (password_verify).
- **Modul Register (register.php):** Menangani pendaftaran akun pengguna baru mencakup penampungan Nama, Alamat Email, Username, Password, dan Re-Password.

1.2 Perbaikan Kode (Refactoring & Fix Bug)

Dalam proses analisis awal terhadap kode sumber repositori target, ditemukan beberapa kesalahan kode yang telah diperbaiki:

1. **Koreksi Variabel pada register.php:** Pada query INSERT INTO users di repositori asli, digunakan variabel \$nama yang belum terdefinisi. Variabel tersebut diperbaiki menjadi \$name.
2. **Penyelarasan Verifikasi Username:** Pemanggilan fungsi cek_nama() disesuaikan untuk memeriksa ketersediaan \$username pada database secara tepat.
3. **Pembuatan Halaman Landing Page (index.php):** Repositori awal belum menyertakan file index.php yang dituju setelah login/register sukses. Dibat halaman dashboard sederhana yang menampilkan pesan selamat datang dan tombol Logout.

BAB II: PERANCANGAN TEST CASE

Berikut adalah matriks test case yang dirancang untuk menguji fungsionalitas positif dan negatif pada modul Login dan Register:

ID	Modul	Deskripsi Skenario	Input Data Test	Ekspektasi Hasil	Hasil Test
TC_REG_01	Register	Registrasi akun baru dengan data valid lengkap	Name: User Valid Email: u1@test.com User: uvalid1 Pass: pass123	Registrasi berhasil, redirect ke index.php	PASSED
TC_REG_02	Register	Registrasi dengan Password & Re-Password tidak cocok	Pass: pass123 RePass: pass999	Gagal, error: <i>Password tidak sama !!</i>	PASSED
TC_REG_03	Register	Registrasi dengan form kosong	Form dikirim kosong	Gagal, alert: <i>Data tidak boleh kosong !!</i>	PASSED
TC_REG_04	Register	Registrasi dengan Username yang sudah terdaftar	Username: uvalid1 (duplikat)	Gagal, alert: <i>Username sudah terdaftar !!</i>	PASSED
TC_LOG_01	Login	Login pengguna dengan kredensial valid	User: uvalid1 Pass: pass123	Login berhasil, redirect ke index.php	PASSED
TC_LOG_02	Login	Login dengan Password salah	User: uvalid1 Pass: wrongpass	Login gagal, tetap berada di login.php	PASSED
TC_LOG_03	Login	Login dengan Username tidak terdaftar	User: user_unknown Pass: pass123	Gagal, alert: <i>Register User Gagal !!</i>	PASSED
TC_LOG_04	Login	Login dengan form kosong	Form dikirim kosong	Gagal, alert: <i>Data tidak boleh kosong !!</i>	PASSED

BAB III: IMPLEMENTASI STUB DAN DRIVER

3.1 Konsep dan Peran Stub

Dalam pengujian perangkat lunak, **Stub** adalah komponen pasif / simulasi tiruan yang digunakan untuk menggantikan modul dependen yang belum siap atau kompleks untuk dihubungkan secara penuh.

Pada praktikum ini, Stub diterapkan pada dua area:

1. **Database Stub / Auto-Provisioner (koneksi_stub.php / koneksi.php)**: Menyediakan mekanisme pengganti koneksi database dan skema otomatis sehingga pengujian tidak mengalami kegagalan akibat ketiadaan tabel users.
2. **Dashboard Target Stub (index.php)**: Bertindak sebagai komponen penerima pengalihan (redirect stub) untuk memvalidasi bahwa sesi autentikasi berhasil terbentuk.

3.2 Konsep dan Peran Driver

Driver (Selenium WebDriver) adalah komponen aktif pengujian yang mengontrol perilaku peramban web secara otomatis. Driver bertindak seolah-olah pengguna nyata sedang berinteraksi dengan aplikasi.

BAB IV: PANDUAN EKSEKUSI PENGUJIAN UNTUK DOSEN / PENGUJI

4.1 Prasyarat Lingkungan Pengujian

Sebelum pengujian dijalankan, pastikan sistem memiliki:

- Python 3.8+ (Library: selenium, webdriver-manager)
- PHP 8.x (Terdaftar di Environment PATH)
- XAMPP MySQL (Modul MySQL diaktifkan di XAMPP Control Panel)

4.2 Langkah Menjalankan Pengujian secara Lokal (CLI)

Buka Terminal / PowerShell di direktori proyek, lalu ketik perintah:

```
python -u test_app.py
atau
python -m unittest test_app.py -v
```

4.3 Penjelasan Alur Otomatisasi Script (test_app.py)

1. **setUpClass()**: Script memeriksa port 8000. Jika belum aktif, script menyalakan PHP server (php -S 127.0.0.1:8000) di background secara otomatis.
2. **Inisialisasi Selenium Chrome Driver**: Selenium 4 menginisialisasi peramban Google Chrome headless secara otomatis.
3. **Eksekusi Skenario Test (TC_REG & TC_LOG)**: Selenium mengetikkan teks ke atribut elemen ID, mengklik tombol submit, membaca alert kesalahan, dan memeriksa pengalihan URL.
4. **tearDownClass()**: Setelah pengujian selesai, Selenium menutup browser dan mematikan server PHP.

4.4 Bukti Hasil Output Terminal (Terminal Execution Logs)

```
PS D:\alba\Akademik\Semester 6\Pengujian Perangkat Lunak\Pertemuan 15> python -m unittest test_app.py -v
[+] Memulai inisialisasi lingkungan pengujian...
[+] Menjalankan PHP Built-in Server di http://127.0.0.1:8000 ...
[+] Membuka Selenium Chrome Headless Driver...
[+] Environment siap! Mengoperasikan test suite...

test_TC_LOG_01_login_success (__main__.TestLoginRegister) ... ok
test_TC_LOG_02_wrong_password (__main__.TestLoginRegister) ... ok
test_TC_LOG_03_unregistered_user (__main__.TestLoginRegister) ... ok
test_TC_LOG_04_empty_login_fields (__main__.TestLoginRegister) ... ok
test_TC_REG_01_register_success (__main__.TestLoginRegister) ... ok
test_TC_REG_02_password_mismatch (__main__.TestLoginRegister) ... ok
test_TC_REG_03_empty_fields (__main__.TestLoginRegister) ... ok
test_TC_REG_04_duplicate_username (__main__.TestLoginRegister) ... ok

[+] Membersihkan driver dan menutup server...
[+] Pengujian selesai.
-----
Ran 8 tests in 12.184s

OK
```

BAB V: KODE SUMBER LENGKAP (test_app.py)

Berikut adalah seluruh isi kode sumber file test_app.py (217 baris utuh):

```
import os
import sys
import time
import socket
import subprocess
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

BASE_URL = "http://127.0.0.1:8000"

class TestLoginRegister(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        print("\n[+] Memulai inisialisasi lingkungan pengujian...", flush=True)
        cls.php_process = None

        # Check if PHP server is running on 127.0.0.1:8000
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(0.5)
            res = sock.connect_ex(('127.0.0.1', 8000))
            sock.close()
            if res != 0:
                print("[+] Menjalankan PHP Built-in Server di http://127.0.0.1:8000 ...", flush=True)
                cls.php_process = subprocess.Popen(
                    ["php", "-S", "127.0.0.1:8000"],
                    stdout=subprocess.DEVNULL,
                    stderr=subprocess.DEVNULL
                )
                time.sleep(1.5)
            else:
                print("[+] PHP Server sudah aktif di http://127.0.0.1:8000", flush=True)
        except Exception as e:
            print(f"[!] Warning server check: {e}", flush=True)

        print("[+] Membuka Selenium Chrome Headless Driver...", flush=True)
        chrome_options = Options()
        chrome_options.add_argument("--headless=new")
        chrome_options.add_argument("--no-sandbox")
        chrome_options.add_argument("--disable-dev-shm-usage")
        chrome_options.add_argument("--disable-gpu")
        chrome_options.add_argument("--window-size=1920,1080")

        # Selenium 4 automatically handles chromedriver management natively
        cls.driver = webdriver.Chrome(options=chrome_options)
        cls.driver.implicitly_wait(5)
        print("[+] Environment siap! Mengoperasikan test suite...\n", flush=True)

    @classmethod
    def tearDownClass(cls):
        print("\n[+] Membersihkan driver dan menutup server...", flush=True)
        if hasattr(cls, 'driver') and cls.driver:
            cls.driver.quit()
        if cls.php_process:
            cls.php_process.terminate()
        print("[+] Pengujian selesai.", flush=True)

    def setUp(self):
        # Clear cookies before each test to reset session
        self.driver.get(f"{BASE_URL}/login.php")
        self.driver.delete_all_cookies()

    # =====
    # MODUL REGISTER TEST CASES
    # =====

    def test_TC_REG_01_register_success(self):
        """TC_REG_01: Registrasi berhasil dengan data valid"""
        self.driver.get(f"{BASE_URL}/register.php")
        test_user = f"user_{int(time.time())}"

        self.driver.find_element(By.ID, "name").send_keys("Pengguna Test Valid")
        self.driver.find_element(By.ID, "inputEmail").send_keys(f"{test_user}@example.com")
        self.driver.find_element(By.ID, "username").send_keys(test_user)
        self.driver.find_element(By.ID, "InputPassword").send_keys("password123")
        self.driver.find_element(By.ID, "InputRePassword").send_keys("password123")
```

```

        self.driver.find_element(By.NAME, "submit").click()
        time.sleep(1)

        self.assertIn("index.php", self.driver.current_url)
        welcome_elem = self.driver.find_element(By.ID, "welcome-user")
        self.assertEqual(welcome_elem.text, test_user)

def test_TC_REG_02_password_mismatch(self):
    """TC_REG_02: Registrasi gagal - Password dan Re-Password tidak cocok"""
    self.driver.get(f"{BASE_URL}/register.php")

    self.driver.find_element(By.ID, "name").send_keys("User Beda Password")
    self.driver.find_element(By.ID, "inputEmail").send_keys("bedapass@example.com")
    self.driver.find_element(By.ID, "username").send_keys("bedapassuser")
    self.driver.find_element(By.ID, "InputPassword").send_keys("password123")
    self.driver.find_element(By.ID, "InputRePassword").send_keys("password999")

    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    error_msg = self.driver.find_element(By.CLASS_NAME, "text-danger").text
    self.assertIn("Password tidak sama", error_msg)

def test_TC_REG_03_empty_fields(self):
    """TC_REG_03: Registrasi gagal - Data tidak boleh kosong"""
    self.driver.get(f"{BASE_URL}/register.php")
    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    alert = self.driver.find_element(By.CLASS_NAME, "alert-danger").text
    self.assertIn("Data tidak boleh kosong", alert)

def test_TC_REG_04_duplicate_username(self):
    """TC_REG_04: Registrasi gagal - Username sudah terdaftar"""
    dup_username = "userduplikasi"
    self.driver.get(f"{BASE_URL}/register.php")
    self.driver.find_element(By.ID, "name").send_keys("User Pertama")
    self.driver.find_element(By.ID, "inputEmail").send_keys("dup1@example.com")
    self.driver.find_element(By.ID, "username").send_keys(dup_username)
    self.driver.find_element(By.ID, "InputPassword").send_keys("password123")
    self.driver.find_element(By.ID, "InputRePassword").send_keys("password123")
    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    self.driver.get(f"{BASE_URL}/index.php?action=logout")
    time.sleep(1)

    self.driver.get(f"{BASE_URL}/register.php")
    self.driver.find_element(By.ID, "name").send_keys("User Kedua")
    self.driver.find_element(By.ID, "inputEmail").send_keys("dup2@example.com")
    self.driver.find_element(By.ID, "username").send_keys(dup_username)
    self.driver.find_element(By.ID, "InputPassword").send_keys("password123")
    self.driver.find_element(By.ID, "InputRePassword").send_keys("password123")
    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    alert = self.driver.find_element(By.CLASS_NAME, "alert-danger").text
    self.assertIn("Username sudah terdaftar", alert)

# =====
# MODUL LOGIN TEST CASES
# =====

def test_TC_LOG_01_login_success(self):
    """TC_LOG_01: Login berhasil dengan data valid"""
    user = "logininvaliduser"
    password = "secretpassword"

    self.driver.get(f"{BASE_URL}/register.php")
    self.driver.find_element(By.ID, "name").send_keys("Login Valid")
    self.driver.find_element(By.ID, "inputEmail").send_keys("logininvalid@example.com")
    self.driver.find_element(By.ID, "username").send_keys(user)
    self.driver.find_element(By.ID, "InputPassword").send_keys(password)
    self.driver.find_element(By.ID, "InputRePassword").send_keys(password)
    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    self.driver.get(f"{BASE_URL}/index.php?action=logout")
    time.sleep(1)

    self.driver.get(f"{BASE_URL}/login.php")
    self.driver.find_element(By.ID, "username").send_keys(user)
    self.driver.find_element(By.ID, "InputPassword").send_keys(password)
    self.driver.find_element(By.NAME, "submit").click()

```

```
time.sleep(1)

self.assertIn("index.php", self.driver.current_url)
welcome_elem = self.driver.find_element(By.ID, "welcome-user")
self.assertEqual(welcome_elem.text, user)

def test_TC_LOG_02_wrong_password(self):
    """TC_LOG_02: Login gagal - Password salah"""
    user = "passssalahuser"
    password = "correctpassword"

    self.driver.get(f"{BASE_URL}/register.php")
    self.driver.find_element(By.ID, "name").send_keys("Pass Salah")
    self.driver.find_element(By.ID, "inputEmail").send_keys("passssalah@example.com")
    self.driver.find_element(By.ID, "username").send_keys(user)
    self.driver.find_element(By.ID, "InputPassword").send_keys(password)
    self.driver.find_element(By.ID, "InputRePassword").send_keys(password)
    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    self.driver.get(f"{BASE_URL}/index.php?action=logout")
    time.sleep(1)

    self.driver.get(f"{BASE_URL}/login.php")
    self.driver.find_element(By.ID, "username").send_keys(user)
    self.driver.find_element(By.ID, "InputPassword").send_keys("wrongpassword")
    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    self.assertIn("login.php", self.driver.current_url)

def test_TC_LOG_03_unregistered_user(self):
    """TC_LOG_03: Login gagal - Username tidak terdaftar"""
    self.driver.get(f"{BASE_URL}/login.php")
    self.driver.find_element(By.ID, "username").send_keys("nonexistentuser999")
    self.driver.find_element(By.ID, "InputPassword").send_keys("anypassword")
    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    alert = self.driver.find_element(By.CLASS_NAME, "alert-danger").text
    self.assertIn("Register User Gagal", alert)

def test_TC_LOG_04_empty_login_fields(self):
    """TC_LOG_04: Login gagal - Data tidak boleh kosong"""
    self.driver.get(f"{BASE_URL}/login.php")
    self.driver.find_element(By.NAME, "submit").click()
    time.sleep(1)

    alert = self.driver.find_element(By.CLASS_NAME, "alert-danger").text
    self.assertIn("Data tidak boleh kosong", alert)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

BAB VI: KONFIGURASI CI/CD PIPELINE (GITHUB ACTIONS)

File workflow `.github/workflows/selenium-tests.yml` yang terpasang di GitHub Actions:

```
name: Selenium Testing & CI/CD Pipeline

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

jobs:
  selenium-tests:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ALLOW_EMPTY_PASSWORD: 'yes'
          MYSQL_DATABASE: quiz_pengupil
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=10

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.2'
          extensions: mysqli, pdo, pdo_mysql

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install Python Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install selenium webdriver-manager pytest

      - name: Install MySQL Client & Import Schema
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y mysql-client
          mysql -h 127.0.0.1 -u root --protocol=tcp quiz_pengupil < db/quiz_pengupil.sql || true

      - name: Start PHP Built-in Web Server
        run: |
          php -S 127.0.0.1:8000 &
          sleep 3

      - name: Run Selenium Test Suite
        env:
          DB_HOST: '127.0.0.1'
          DB_USER: 'root'
          DB_PASS: ''
          DB_NAME: 'quiz_pengupil'
        run: |
          python -u test_app.py
```

BAB VII: KESIMPULAN & LINK REPOSITORY

Seluruh 8 test case untuk modul Login dan Register telah teruji **100% PASSED**. Pengujian terotomatisasi dengan Selenium WebDriver dan CI/CD GitHub Actions berjalan lancar.

Link Repository GitHub Target / Pengerjaan: <https://github.com/hermanka/quiz-pengupil>